

MLE Sheet 9 Task 1.

1.1. Proof: Easy to show as

$$k_1(x, x') \geq 0 \wedge k_2(x, x') \geq 0$$

$$\Rightarrow k_1(x, x') + k_2(x, x') \geq 0. \text{ Thus } k_{\text{sum}} \text{ psd } \square.$$

1.2. (a) For the GP, we have the assumption that: (K_θ is the approximation to covariance matrix)
 $f(x) \sim N(0, K_\theta)$, ($m(x) = 0$)
 From the task, $\varepsilon \sim N(0, \sigma^2 I_n)$ was given.

Hence, $y|x \sim f(x) + \varepsilon \sim N(0, K_\theta + \sigma^2 I_n)$.

$$p(y|x, \theta) = \frac{1}{\sqrt{(2\pi)^n |K_\theta + \sigma^2 I_n|}} \exp\left(-\frac{1}{2} y^T (K_\theta + \sigma^2 I_n)^{-1} y\right)$$

which gives the log-likelihood as stated:

$$\log p(y|x, \theta) = -\frac{1}{2} y^T (K_\theta + \sigma^2 I_n)^{-1} y - \frac{1}{2} \log |K_\theta + \sigma^2 I_n| - \frac{n}{2} \log(2\pi)$$

(b) Data Fit: $-\frac{1}{2} y^T (K_\theta + \sigma^2 I_n)^{-1} y$.

The better K_θ approximates the real covariance matrix, the ~~better~~ closer to 0 is this term for the training data.

Complexity Penalty: $-\frac{1}{2} \log |K_\theta + \sigma^2 I_n|$.

If K_θ is too complicated (too many nonzero entries; entries too large), this penalty term will punish the complexity by making the log-likelihood smaller.

Why Occam's razor?: Occam's Razor encourages simple models to avoid overfitting, which agrees with the second term.

mle9-task1.3

July 6, 2025

1 Machine Learning Essentials SS25 - Exercise Sheet 9, Task 1.3

1.1 Instructions

- TODO's indicate where you need to complete the implementations.
- You may use external resources, but write your own solutions.
- Provide concise, but comprehensible comments to explain what your code does.
- Code that's unnecessarily extensive and/or not well commented will not be scored.

Note: This exercise was inspired by Section 5.4.3 of the “Bible” of GPs: “Gaussian Processes for Machine Learning” by Rasmussen & Williams. It's freely available here: <https://gaussianprocess.org/gpml/chapters/RW.pdf>

```
[ ]: import torch
import gpytorch # pip install gpytorch
import numpy as np
import pandas as pd # pip install pandas, pandas is a popular library for data
    ↳ handling, especially for tabular data
import matplotlib.pyplot as plt
from tqdm.notebook import tqdm # pip install tqdm, for progress bars

import os
# You might want to limit the number of threads used:
os.environ["OMP_NUM_THREADS"] = "10"
os.environ["MKL_NUM_THREADS"] = "10"

# If you have a GPU available, you should move your tensors to the GPU for
    ↳ (much) faster computation:
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

torch.manual_seed(42)
```

Note: you may need to restart the kernel to use updated packages.

Using device: cpu

```
[ ]: <torch._C.Generator at 0x126010a90>
```

```
[3]: # Load the data using pandas, remove missing values from the averages
co2_data = pd.read_csv('https://gml.noaa.gov/webdata/ccgg/trends/co2/co2_mm_mlo.
    ↪ csv', sep=',', header=40, na_values="-999.99").dropna()
```

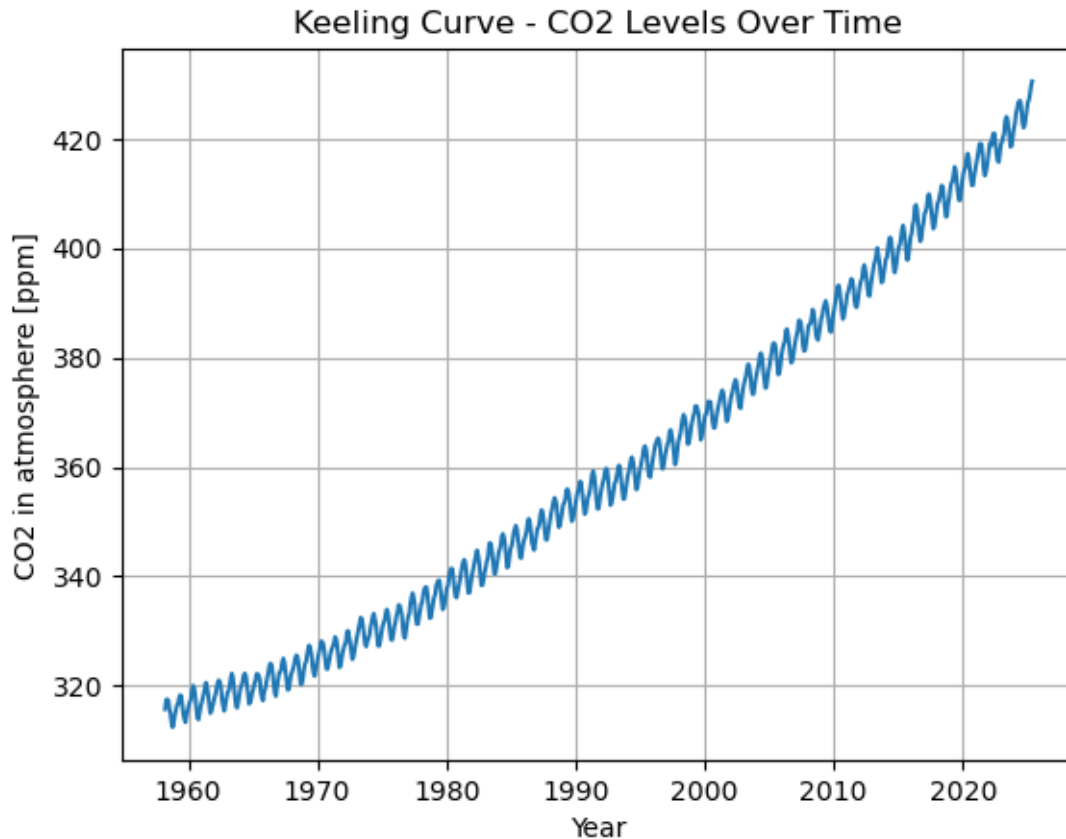
```
[ ]: # We'll only need 2 relevant columns: decimal date and average CO2
X = co2_data['decimal date'].values
y = co2_data['average'].values

# Convert to PyTorch tensors
X_tensor = torch.tensor(X, dtype=torch.float32)
y_tensor = torch.tensor(y, dtype=torch.float32)

X_train = X_tensor
y_train = y_tensor
plt.plot(X_train.numpy(), y_train.numpy())
plt.xlabel('Year')
plt.ylabel('CO2 in atmosphere [ppm]')
plt.title('Keeling Curve - CO2 Levels Over Time')
plt.grid()
plt.show()

print(f"{len(X_train)}") # Number of training points

# Standardize the data (helps with stability and convergence of training the GP
    ↪ model)
X_mean, X_std = X_tensor.mean(), X_tensor.std()
y_mean, y_std = y_tensor.mean(), y_tensor.std()
X_train = (X_tensor - X_mean) / X_std
y_train = (y_tensor - y_mean) / y_std
```



807

```
[ ]: tensor(19.4262)
```

```
[24]: # Define a GP model using GPyTorch
class CO2GPMModel(gpytorch.models.ExactGP):
    def __init__(self, train_x, train_y, likelihood):
        super().__init__(train_x, train_y, likelihood)
        # Using a linear mean function makes sense here, since we expect a
        ↪ long-term trend in the CO2 data.
        self.mean_module = gpytorch.means.LinearMean(input_size=1)

        # TODO: This is the core of the task. You need to design a composite
        ↪ kernel that captures the different patterns observed in the time series
        ↪ (think about where these could come from!), e.g.
        # 1. A long-term, slowly increasing trend.
        # 2. A seasonal (periodic) component.
        # 3. A medium-term component.
        # 3. Smaller, short-term irregularities or noise.
```

```

    # At the end, you want to combine these by adding/multiplying and/or
    ↪ scaling them (using gpytorch.kernels.ScaleKernel). Refer e.g. to the Kernel
    ↪ Cookbook, or the textbook mentioned above for inspiration. Choices here
    ↪ could include `RBFKernel`, `PeriodicKernel`, `MaternKernel`, `RQKernel`, and
    ↪ `ScaleKernel`.

    # A possible example could look like this:
    k_trend      = gpytorch.kernels.RBFKernel()
    k_seasonal   = gpytorch.kernels.PeriodicKernel() * gpytorch.kernels.
    ↪ RBFKernel()
    k_mid        = gpytorch.kernels.MaternKernel(nu=1.5)

    # For each kernel you define, you can also set the `outputscale`
    ↪ parameter (e.g. k_long.outputscale) to control the amplitude of the kernel.
    ↪ Choose this based on your expectations about the data.

    self.covar_module = gpytorch.kernels.ScaleKernel(k_trend + k_seasonal +
    ↪ k_mid) # A sum of the kernels with an overall scaling factor
    self.covar_module.outputscale = 0.1

    def forward(self, x):
        # This defines the Prior of the GP. model(new_data) will then condition
        ↪ on some new data to produce the predictive posterior - GPyTorch nicely
        ↪ handles the posterior inference automatically for us.
        mean_x = self.mean_module(x)
        covar_x = self.covar_module(x)
        return gpytorch.distributions.MultivariateNormal(mean_x, covar_x)

```

```

[28]: # Initialize the model and the likelihood
likelihood = gpytorch.likelihoods.GaussianLikelihood() # This incorporates a
    ↪ white noise term into the model (observation noise)
model = CO2GPMModel(X_train, y_train, likelihood)

def initialize_model_params(model, X_std):
    with torch.no_grad():
        kernels = model.covar_module.base_kernel.kernels

        k_trend = kernels[0]
        k_trend.lengthscale = 100.0 / X_std

        k_seasonal = kernels[1]
        periodic_kernel = k_seasonal.kernels[0]
        envelope_rbf = k_seasonal.kernels[1]

        periodic_kernel.period_length = 1.0 / X_std
        periodic_kernel.lengthscale = 0.5 / X_std
        envelope_rbf.lengthscale = 10.0 / X_std

```

```

k_medium = kernels[2]
k_medium.lengthscale = 3.0 / X_std

```

```

model.likelihood.noise = 0.1

```

```

# Initialize the model using your prior knowledge
initialize_model_params(model, X_std)

```

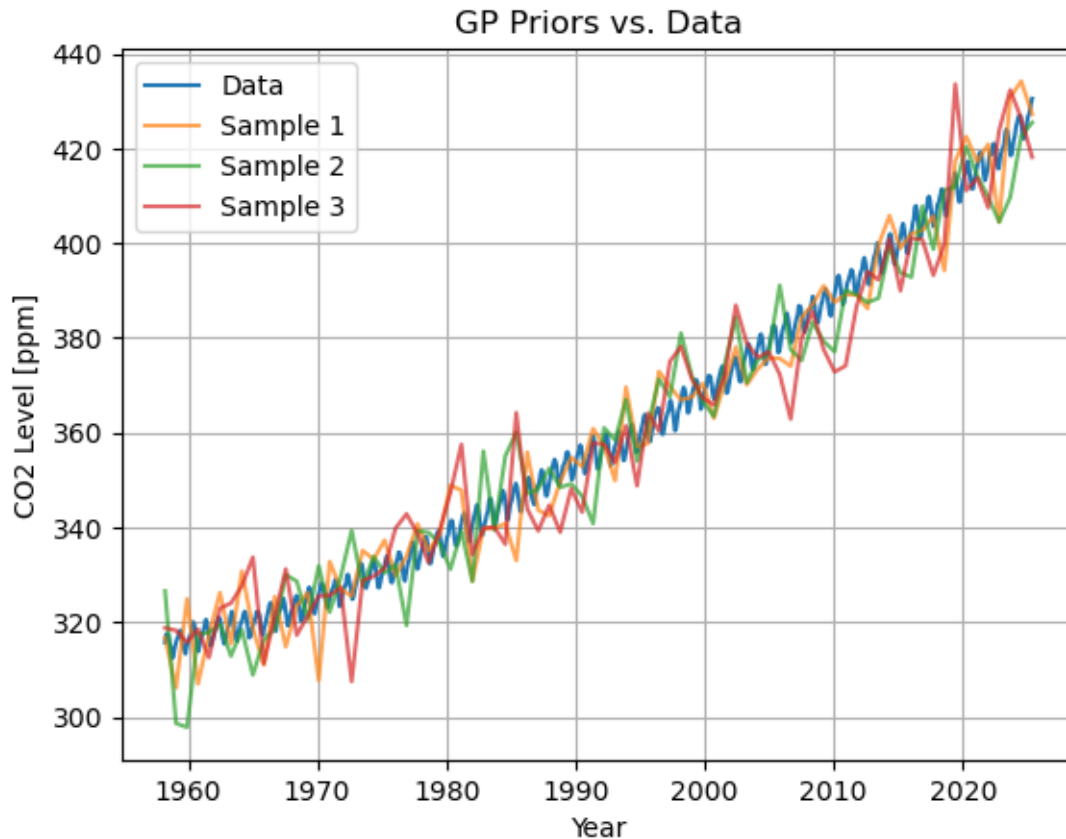
```

[30]: # You can test whether our kernel & initialization makes sense by sampling from
      ↪ the prior distribution a few times and comparing to the data.
model.eval()
# Create a grid for plotting priors
X_prior_plot = torch.linspace(X_train.min(), X_train.max(), 80)
with torch.no_grad(), gpytorch.settings.fast_pred_var():
    prior_samples = model(X_prior_plot).sample(torch.Size([3])) # Sample 3
    ↪ functions from the prior, evaluated on the grid

# Un-standardize the prior samples and X coordinates for plotting in the
  ↪ original coordinates
prior_samples_unscaled = prior_samples * y_std + y_mean
X_prior_plot_unscaled = X_prior_plot * X_std + X_mean

# Plot the prior samples over the data
plt.plot(X_tensor.numpy(), y_tensor.numpy(), label='Data')
for i in range(prior_samples.shape[0]):
    plt.plot(X_prior_plot_unscaled.numpy(), prior_samples_unscaled[i].numpy(),
    ↪ label=f'Sample {i+1}', alpha=0.7)
plt.title('GP Priors vs. Data')
plt.xlabel('Year')
plt.ylabel('CO2 Level [ppm]')
plt.legend()
plt.grid()
plt.show()

```



```
[31]: # For optimizing the model, put the model and likelihood in training mode
model.train()
likelihood.train()

# We'll use SGD with the Adam optimizer
# TODO: Set a good learning rate for Adam.
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

# The optimization criterion is the marginal log likelihood, as described in Task 1.2
mll = gpytorch.mlls.ExactMarginalLogLikelihood(likelihood, model)

# Training loop
num_epochs = 300 # adjust as needed
loss_history = []
# Optimize log marginal likelihood
for i in range(num_epochs):
    # TODO: Implement the training loop
    # 1. Zero the gradients
    # 2. Get output from model
```

```

# 3. Calculate the loss (= negative MLL), loss = ...
# 4. Backpropagate the loss, perform optimizer step
optimizer.zero_grad()
output = model(X_train)
loss = -mll(output, y_train)
loss.backward()
optimizer.step()

loss_history.append(loss.item()) # store loss values for plotting
if (i + 1) % 25 == 0:
    print(f'Iter {i+1}/{num_epochs} - Loss: {loss.item():.3f}') # print
    ↪progress

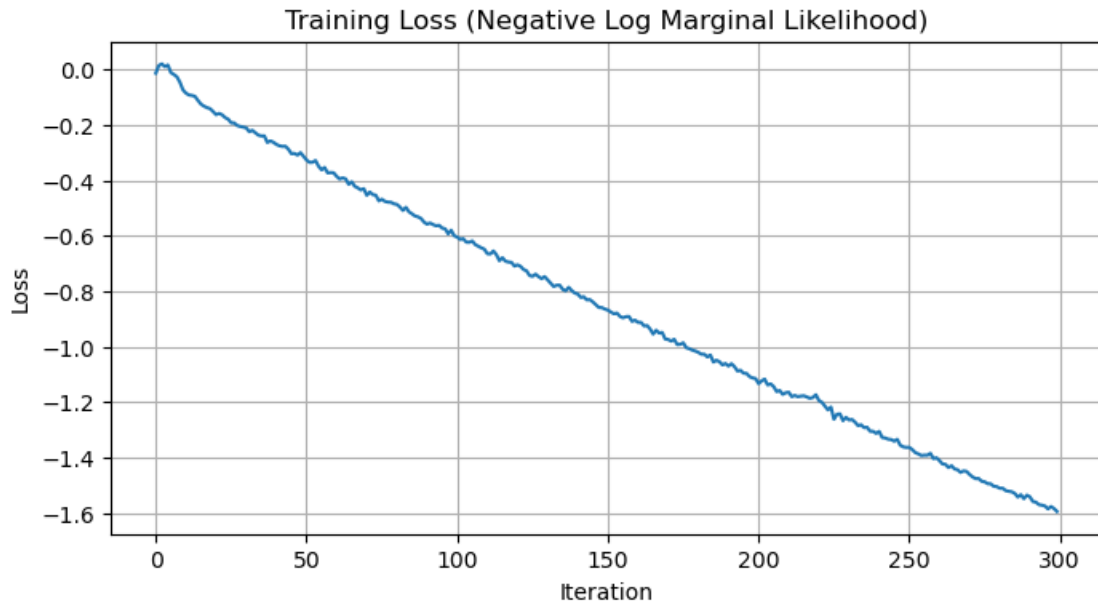
# Plot training loss
plt.figure(figsize=(8, 4))
plt.plot(loss_history)
plt.title('Training Loss (Negative Log Marginal Likelihood)')
plt.xlabel('Iteration')
plt.ylabel('Loss')
plt.grid()
plt.show()

```

```

Iter 25/300 - Loss: -0.179
Iter 50/300 - Loss: -0.313
Iter 75/300 - Loss: -0.474
Iter 100/300 - Loss: -0.600
Iter 125/300 - Loss: -0.743
Iter 150/300 - Loss: -0.865
Iter 175/300 - Loss: -0.992
Iter 200/300 - Loss: -1.116
Iter 225/300 - Loss: -1.218
Iter 250/300 - Loss: -1.364
Iter 275/300 - Loss: -1.486
Iter 300/300 - Loss: -1.595

```

```
[32]: # Plotting results

# Put the model in evaluation mode
model.eval()
likelihood.eval()

# Create some test points, including extrapolation into the future
X_test = torch.linspace(1950, 2045, 500) # Predict from 1950 to 2045 - adjust
↳as you like
X_test_scaled = (X_test - X_mean) / X_std

# Make predictions and get credible intervals
with torch.no_grad(), gpytorch.settings.fast_pred_var(): #fast_pred_var speeds
↳up the predictions by using a fast approximation of the predictive variance
    predictions = likelihood(model(X_test_scaled)) # get posterior predictive
↳distribution
    mean = predictions.mean
    lower, upper = predictions.confidence_region()

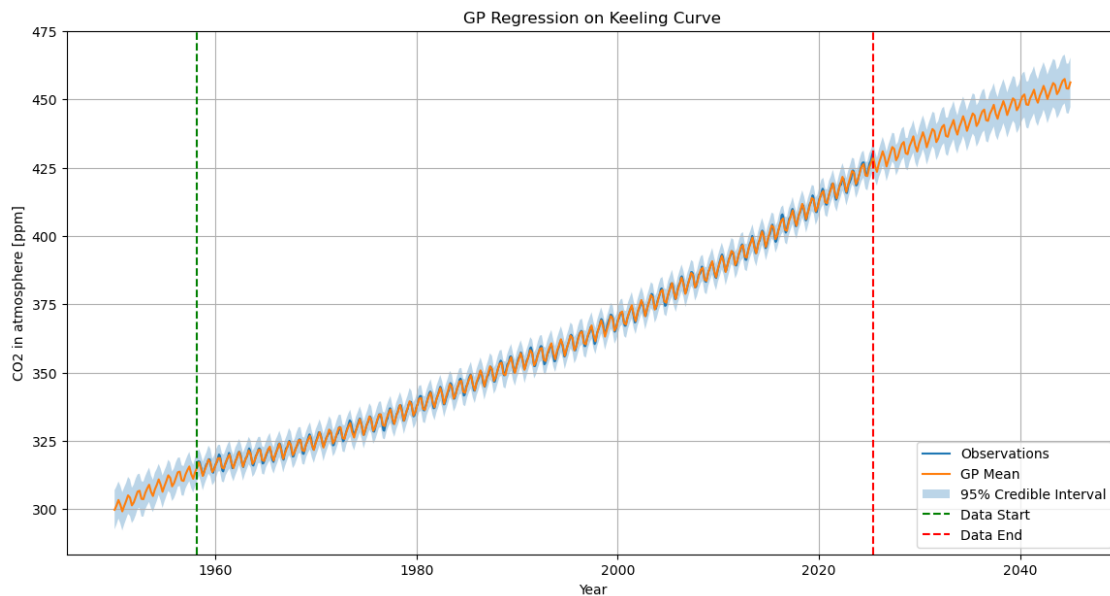
# Un-standardize the results for plotting
mean_unscaled = mean * y_std + y_mean
lower_unscaled = lower * y_std + y_mean
upper_unscaled = upper * y_std + y_mean

plt.figure(figsize=(14, 7))
# Training data
```

```

plt.plot(X, y, markersize=4, label='Observations')
# Predictive mean
plt.plot(X_test.numpy(), mean_unscaled.numpy(), label='GP Mean')
# Confidence region (95% credible interval)
plt.fill_between(X_test.numpy(), lower_unscaled.numpy(), upper_unscaled.
    ↳numpy(), alpha=0.3, label='95% Credible Interval')
# Draw 2 lines where extrapolation starts
last_year = X_train.max().item()
last_year_unscaled = last_year * X_std + X_mean
first_year = X_train.min().item()
first_year_unscaled = first_year * X_std + X_mean
plt.axvline(x=first_year_unscaled, color='green', linestyle='--', label='Data_
    ↳Start')
plt.axvline(x=last_year_unscaled, color='red', linestyle='--', label='Data End')
plt.title('GP Regression on Keeling Curve')
plt.xlabel('Year')
plt.ylabel('CO2 in atmosphere [ppm]')
plt.legend()
plt.grid()
plt.show()

```



[39]:

```

# TODO: Print the learned hyperparameters of your model.
# You can access those as follows:
# model.likelihood.noise.item() # For the likelihood noise
# model.mean_module.weights.item() # For the linear mean function slope
# model.mean_module.bias.item() # For the linear mean function bias

```

```

# model.covar_module.base_kernel.kernels[i].lengthscale.item() # For the
↳ lengthscale of the i-th kernel (or model.covar_module.base_kernel.kernels[i].
↳ period_length.item() for periodic kernels)
print("Learned Hyperparameters:")
print(f"Likelihood noise: {model.likelihood.noise.item():.4f}")
kernels = model.covar_module.base_kernel.kernels
k_trend = kernels[0]
print(f"trend lengthscale: {k_trend.lengthscale.item():.4f}")

k_seasonal = kernels[1]
periodic = k_seasonal.kernels[0]
envelope = k_seasonal.kernels[1]
print(f"seasonal period_length: {periodic.period_length.item():.4f}")
print(f"seasonal periodic lengthscale: {periodic.lengthscale.item():.4f}")
print(f"seasonal envelope lengthscale: {envelope.lengthscale.item():.4f}")

k_mid = kernels[2]
print(f"mid-term lengthscale: {k_mid.lengthscale.item():.4f}")

```

```

Learned Hyperparameters:
Likelihood noise: 0.0050
trend lengthscale: 4.9263
seasonal period_length: 0.0514
seasonal periodic lengthscale: 0.0964
seasonal envelope lengthscale: 1.8823
mid-term lengthscale: 0.7226

```

1.1.1 Results Assessment

TODO: Given your results, assess the performance of your regression model and interpret the learned hyperparameters. Do they coincide with your expectations of the Keeling curve?

The learned CO₂ curve fits the Keeling curve well. The learnt parameters agree perfectly with my expectations. $X_{std} \times \text{PeriodicLength}$ is close to 1(year). The learnt noise is small, which indicates the precision of Keeling's measurement.

mle9-task3

July 6, 2025

1 Machine Learning Essentials SS25 - Exercise Sheet 9

1.1 Proposed Solutions - Exercise 3

```
[1]: import torch
import gpytorch
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from IPython.display import HTML
from scipy.ndimage import gaussian_filter
from tqdm import tqdm
from gpytorch.kernels import RBFKernel, ScaleKernel
from gpytorch.means import Mean
from torch.optim import Adam
```

```
[2]: # Search Area Config
GRID_SIZE = 50
# "Secret" location of the wreckage.
TRUE_WRECKAGE_LOCATION = (42, 42)
# Center of our initial prior belief.
PRIOR_CENTER = (25, 25)
PRIOR_WIDTH = 10.0
# Create coordinate grid of the "ocean" to search.
xx, yy = np.meshgrid(np.arange(GRID_SIZE), np.arange(GRID_SIZE))
grid_coords = torch.tensor(np.vstack([xx.ravel(), yy.ravel()]).T, dtype=torch.
    ↪float)
```

```
[3]: # Search environment
# You're given 2 key components:
# The prior belief map, which is a 2D Gaussian blob representing our initial,
    ↪uncertain guess about the wreckage location. This will be used as the prior
    ↪mean for our GP.
# The search function, which is a function that simulates a noisy sonar
    ↪measurement. It returns a strong signal near the wreckage and a weak, noisy
    ↪signal far away.

def create_prior_map(center, width):
```



```

"""Creates a 2D Gaussian blob as our prior belief."""
dist_sq = (xx - center[0])**2 + (yy - center[1])**2
prior_map = np.exp(-dist_sq / (2 * width**2))
return torch.tensor(prior_map, dtype=torch.float)

def create_search_function(wreckage_loc=TRUE_WRECKAGE_LOCATION, amplitude=10.0,
↪signal_width=4.0, noise_level=0.5):
    " Create a noisy search function with a hidden target."
    true_x, true_y = wreckage_loc

    def search(x, y):
        """Simulates a noisy sonar reading at a given coordinate."""
        dist_sq = (x - true_x)**2 + (y - true_y)**2
        signal = amplitude * np.exp(-dist_sq / (2 * signal_width**2))
        noise = np.random.normal(0, noise_level) # Measurement noise
        return signal + noise

    return search

prior_belief_map = create_prior_map(PRIOR_CENTER, PRIOR_WIDTH)
perform_search = create_search_function(TRUE_WRECKAGE_LOCATION)

```

```

[4]: class PriorMean(gpytorch.means.Mean):
    """A GP mean function that uses a pre-defined prior map."""
    def __init__(self, prior_map):
        super().__init__()
        self.register_buffer('prior_map', prior_map) # This allows the prior
↪map to be part of the model's state, so it's accessible during training and
↪inference.

    def forward(self, input):
        # TODO: Define a custom mean function for your GP that uses the prior
↪belief map.
        x = input[:, 0].clamp(0, self.prior_map.shape[1] - 1)
        y = input[:, 1].clamp(0, self.prior_map.shape[0] - 1)
        x_idx = x.round().long()
        y_idx = y.round().long()
        return self.prior_map[y_idx, x_idx]

class BayesianSearchGP(gpytorch.models.ExactGP):
    """Our GP model for the search, incorporating the prior belief map."""
    def __init__(self, train_x, train_y, likelihood, prior_map):
        super().__init__(train_x, train_y, likelihood)
        self.mean_module = PriorMean(prior_map)
        self.covar_module = gpytorch.kernels.ScaleKernel(gpytorch.kernels.
↪RBFKernel())

```

```

def forward(self, x):
    mean_x = self.mean_module(x)
    covar_x = self.covar_module(x)
    return gpytorch.distributions.MultivariateNormal(mean_x, covar_x)

```

[5]: *# TODO: Implement an acquisition function for the Bayesian search.*

```

def acquisition_function(X, gp, beta=2.0):
    mu, sigma = gp.predict(X, return_std=True)
    return mu + beta * sigma

```

[6]: *# Bayesian Search loop*

This is the main loop where the agent iteratively searches the ocean.

In each step:

1. Fit the GP to all observations made so far.

2. Use the UCB acquisition function to decide where to search next.

3. Call `perform\_search` to get a new observation.

4. Add the new data to the agent's dataset.

Initialization

Start with no data

```
train_x = torch.empty(0, 2, dtype=torch.float)
```

```
train_y = torch.empty(0, dtype=torch.float)
```

TODO: Instantiate the GP model (with the prior belief map), the

↪ GaussianLikelihood, and the optimizer (use e.g. Adam with an appropriate

↪ learning rate).

```
likelihood = gpytorch.likelihoods.GaussianLikelihood()
```

```
model = BayesianSearchGP(train_x, train_y, likelihood, prior_belief_map)
```

```
optimizer = Adam(model.parameters(), lr=0.1)
```

TODO: Define the marginal log likelihood (mll) for the GP model.

```
mll = gpytorch.mlls.ExactMarginalLogLikelihood(likelihood, model)
```

Search parameters

```
N_SEARCHES = 30
```

```
search_history = []
```

```
posterior_mean_history = []
```

```
for i in tqdm(range(N_SEARCHES)):
```

```
    if i > 0:
```

```
        model.train()
```

```
        likelihood.train()
```

```
        for j in range(50): # TODO: Fit the GP model for a few epochs
```

```
            # --> Zero gradients, forward pass, calculate loss, backward pass,
```

↪ and optimize

```
            optimizer.zero_grad()
```

```
            output = model(train_x)
```

```

        loss = -mll(output, train_y)
        loss.backward()
        optimizer.step()

    model.eval()
    likelihood.eval()
    with torch.no_grad():
        # TODO: Select the next point to search using your acquisition function.
        ↪ Then, choose the coordinates of the next point to search.
        posterior = model(grid_coords)
        mu = posterior.mean
        sigma = posterior.variance.sqrt()
        beta = 2.0
        acquisition_function_values = mu + beta * sigma
        next_point_idx = torch.argmax(acquisition_function_values)
        next_point_coors = grid_coords[next_point_idx]

        # TODO: Perform the search on the "candidate" point, chosen by your
        ↪ acquisition function
        x, y = next_point_coors.tolist()
        observation = perform_search(x, y) # TODO: Perform the search

        # Store history for visualization
        search_history.append((x, y, observation))
        with torch.no_grad():
            posterior_mean_history.append(model(grid_coords).mean.view(GRID_SIZE,
            ↪ GRID_SIZE).numpy())

        # 4. Add new data to the dataset
        train_x = torch.cat([train_x, next_point_coors.view(1, 2)])
        train_y = torch.cat([train_y, torch.tensor([observation], dtype=torch.
        ↪ float)])
        model.set_train_data(train_x, train_y, strict=False)

```

100% | 30/30 [00:00<00:00, 32.84it/s]

[8]: # Use the following code to generate an animation of the Bayesian search
 ↪ process. You can play around with the parameters to make it more visually
 ↪ appealing if you like.

```

# Helper function for ocean background
def create_ocean_background(grid_size=50, noise_strength=0.1, noise_scale=5.0):
    gradient = np.tile(np.linspace(0, 1, grid_size), (grid_size, 1)).T
    noise = gaussian_filter(np.random.randn(grid_size, grid_size),
    ↪ sigma=noise_scale)
    return gradient + noise * noise_strength

```

```

# Animation setup
fig, ax = plt.subplots(figsize=(10, 8))
ocean_bg = create_ocean_background(GRID_SIZE)
def animate(frame):
    ax.clear()
    # Plot ocean background
    extent = [-0.5, GRID_SIZE - 0.5, -0.5, GRID_SIZE - 0.5] # Define extent to
    ↪make visualization clearer
    ax.imshow(ocean_bg, cmap='ocean', origin='lower', extent=extent, zorder=-1)

    # Plot the belief map (GP posterior mean)
    belief_map = posterior_mean_history[frame]
    ax.contourf(xx, yy, belief_map, levels=15, cmap='Reds', alpha=0.6, zorder=1)

    # Plot search points
    points = np.array(search_history[:frame+1])
    sc = ax.scatter(points[:, 0], points[:, 1], c=points[:, 2], cmap='viridis',
    ↪s=60, ec='black', zorder=2)

    # Highlight the current search
    ax.scatter(points[-1, 0], points[-1, 1], c='cyan', s=150, ec='black',
    ↪marker='X', zorder=3, label='Current Search')

    # Show the true location
    ax.scatter(
        TRUE_WRECKAGE_LOCATION[0], TRUE_WRECKAGE_LOCATION[1],
        c='lime', s=400, marker='*', ec='black', zorder=4, label='True Wreckage
    ↪Location'
    )

    ax.set_title(f"Bayesian Search: Step {frame + 1}/{N_SEARCHES}", fontsize=16)
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.set_xlim(extent[0], extent[1])
    ax.set_ylim(extent[2], extent[3])
    ax.legend(loc='upper left')
    ax.set_aspect('equal')

ani = FuncAnimation(fig, animate, frames=N_SEARCHES, interval=500, repeat=False)
plt.close(fig) # Prevent static plot from showing before the animation
HTML(ani.to_html5_video())

```

[8]: <IPython.core.display.HTML object>

```

[ ]: # TODO: Visualize the final belief map and search history, compare to true
    ↪wreckage location

```

```

searched_x = [p[0] for p in search_history]
searched_y = [p[1] for p in search_history]

final_posterior_mean = posterior_mean_history[-1]

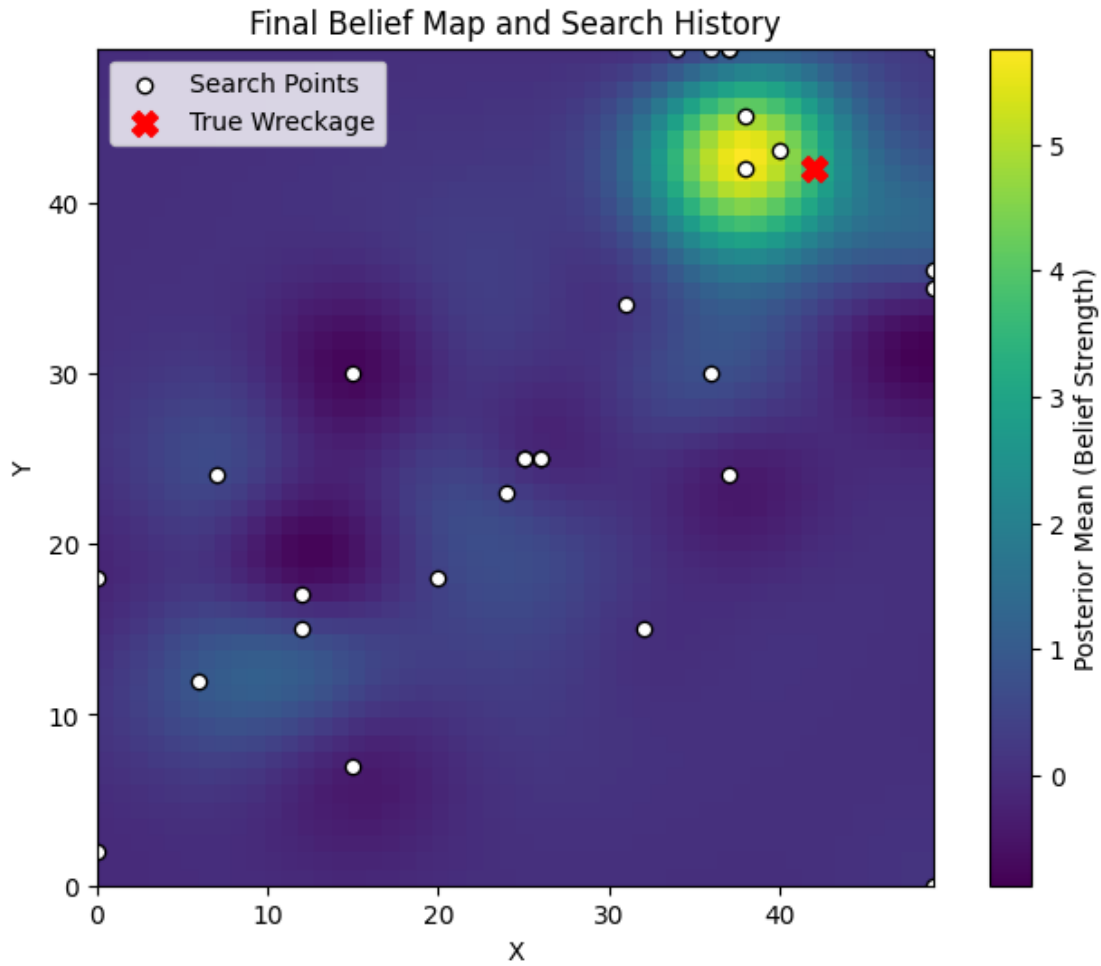
plt.figure(figsize=(8, 6))
plt.imshow(final_posterior_mean, extent=(0, GRID_SIZE-1, 0, GRID_SIZE-1),
           origin='lower', cmap='viridis')
plt.colorbar(label='Posterior Mean (Belief Strength)')
plt.scatter(searched_x, searched_y, c='white', edgecolor='black', label='Search
           Points')
plt.scatter(*TRUE_WRECKAGE_LOCATION, c='red', marker='X', s=100, label='True
           Wreckage')
plt.title("Final Belief Map and Search History")
plt.xlabel("X")
plt.ylabel("Y")
plt.legend()
plt.grid(False)
plt.show()

# TODO: Print the true wreckage location, your search agent's final best guess,
           and the distance between them.
flat_posterior = final_posterior_mean.flatten()
best_guess_idx = np.argmax(flat_posterior)
best_guess_coords = grid_coords[best_guess_idx].tolist()

true_x, true_y = TRUE_WRECKAGE_LOCATION
guess_x, guess_y = best_guess_coords
distance = np.sqrt((true_x - guess_x)**2 + (true_y - guess_y)**2)

print(f"True wreckage location: ({true_x:.2f}, {true_y:.2f})")
print(f"Final best guess:      ({guess_x:.2f}, {guess_y:.2f})")
print(f"Distance error:        {distance:.2f} units")

```

True wreckage location: (42.00, 42.00)
Final best guess: (38.00, 43.00)
Distance error: 4.12 units

TODO: Discuss your results.

In the animation we see that the guesses in the beginning look rather random but they get more precise after we have sonar readings that give us a better approximation for the position.